

ADARE: Learning-Guided Adaptive Control for Many-Objective Workflow Scheduling in Heterogeneous Edge/Fog/Cloud Systems

Roland Cedric Tayo and Sonia Yassa

CY Cergy Paris University, France

rolandcedric.tayo@yahoo.fr, sonia.yassa@cyu.fr

Abstract. Heterogeneous Edge/Fog/Cloud platforms increasingly execute workflows through machine-to-machine coordination, and the difficulty is not only to assign tasks to resources but also to steer the search process as it moves across different trade-off regions.

We present ADARE, a *learning-guided adaptive control mechanism* embedded in a many-objective evolutionary scheduler for heterogeneous Edge/Fog/Cloud workflow execution. Across repeated paired runs, a consistent pattern emerged: fixed operator schedules were often sufficient for early exploration, but they became less reliable once the population entered narrower trade-off regions. ADARE addresses this by treating operator control as an *online learning* problem and adapting crossover behavior during search via a multi-armed bandit controller with an upper-confidence-bound (UCB) rule. The controller is coupled with density-aware mutation and an archive-guided objective refinement layer, while preserving the fairness of the comparison protocol (same initial populations, same evaluation budget, same objective evaluator).

We evaluate ADARE against NSGA-III on three real scientific workflow instances (Montage_25, CyberShake_30, Epigenomics_24) under a heterogeneous Edge/Fog/Cloud resource model. To strengthen statistical validity, we report **20 paired runs per benchmark** and use Wilcoxon signed-rank tests together with Cliff's delta effect sizes. ADARE improves mean core quality metrics (HV/IGD/spacing/epsilon/coverage, excluding runtime) by **26.80%** on average and improves runtime by **5.50%** on average across the three benchmarks. ADARE wins **18/18** quality comparisons (including runtime) and **15/15** core-quality comparisons.

From a systems perspective, ADARE remains practical: the measured upper-bound control overhead (instrumented UCB selection/update and archive routines) is **5.17% on average** over profiled runs, i.e., near the 5% target while improving solution quality. Taken together, these results support viewing ADARE not only as a scheduler variant, but as a reusable online control layer for complex evolutionary search in distributed systems.

Keywords: Online learning · Many-objective optimization · NSGA-III · Multi-armed bandits · Edge/Fog/Cloud · Workflow scheduling

1 Introduction

Modern distributed applications increasingly execute over heterogeneous computing strata, where edge nodes, fog servers, and cloud resources expose markedly different latency, bandwidth, energy, and cost profiles. In this context, workflow scheduling is not only a resource allocation problem; it is also a *control problem under uncertainty* over a search landscape that changes as candidate schedules evolve. We therefore study ADARE as an online control layer embedded in a many-objective evolutionary loop, rather than as a stand-alone scheduler replacement.

NSGA-III remains a strong baseline for many-objective optimization because of its reference-point selection mechanism [6]. However, workflow-scheduling implementations commonly keep crossover and mutation probabilities fixed across the entire run. In heterogeneous Edge/Fog/Cloud settings, this assumption is fragile: an operator that is useful for early exploration can later damage partial structures once the population starts refining narrow trade-off regions. In practice, this can reduce either convergence pressure or diversity exactly when the search distribution starts to shift.

In our early paired runs, this phase-dependent behavior appeared repeatedly in the convergence traces before it became obvious in the summary tables. That pushed us away from redesigning the whole optimizer and toward a more targeted intervention: keep the NSGA-III selection backbone, but let a small online controller adapt crossover choice from observed Pareto-quality rewards. This framing also follows a learning-driven decision view: the optimization engine acts as the environment, operator choice is the action, and search feedback becomes the training signal.

Contributions. The four contributions below came out of repeated implementation iterations in our paired runs and reruns.

1. First, we cast crossover control in ADARE as a *non-stationary multi-armed bandit (MAB)* problem and implement an online UCB controller whose rewards are updated from a Pareto-quality proxy.
2. Second, we add density-aware mutation and archive-guided refinement, while keeping NSGA-III selection fixed for fair comparison.
3. Third, we report a statistically grounded evaluation on three workflow instances with **20 paired runs** per benchmark, and we use Wilcoxon signed-rank tests together with Cliff’s delta effect sizes.
4. Finally, we provide computational complexity analysis and runtime profiling; the extra control layer remains modest relative to schedule evaluation cost.

The paper is organized in the same order the study itself converged: baseline positioning first, then the scheduling model, and finally the control mechanisms retained in the artifact.

2 Related Work

2.1 Workflow Scheduling in Edge/Fog/Cloud Systems

Workflow scheduling in heterogeneous distributed systems is classically formulated as a DAG mapping problem with precedence and communication constraints. Heuristic methods such as HEFT remain competitive for makespan-focused scheduling because of their low computational cost [1], yet they are not naturally designed for multi-objective trade-offs involving latency, energy, and monetary cost. The fog computing paradigm further amplifies this challenge by introducing an intermediate layer with distinct communication and compute properties [2].

For this reason, many-objective formulations are more appropriate in heterogeneous edge, fog, and cloud orchestration than single-score scheduling. The scheduler should expose trade-off solutions rather than a single operating point. In the workflow community, benchmark instances such as Montage, CyberShake, and Epigenomics are widely used to stress both compute and communication behavior; we rely on public workflow repositories and WfCommons-style instances for reproducible experimentation [5].

2.2 Baseline Many-Objective Evolutionary Algorithms

With a moderate objective count, many studies still start from NSGA-II [3]. In our comparisons, NSGA-III keeps the same lineage and uses reference points to retain selection pressure in many-objective settings [6]. MOEA/D [4] is different. It decomposes the problem into coordinated scalar subproblems. We keep this trio because it exposes distinct search biases during runs: dominance ranking, reference-point niching, and decomposition.

In practice, however, workflow scheduling studies often keep crossover and mutation schedules fixed because dynamic operator policies are harder to instrument and compare reproducibly. That engineering convenience becomes a methodological limitation in our setting. ADARE keeps the many-objective selection backbone intact (NSGA-III in the current artifact) and changes the control policy around operator choice and mutation behavior as the search state evolves.

2.3 Learning-Guided Adaptive MOEAs

Adaptive operator selection and reinforcement-learning-assisted evolutionary algorithms have become increasingly relevant in recent years. OVEA introduces Q-learning-driven operator adaptation in a reference-vector-based MOEA [9]. A recent survey by Song et al. synthesizes the broader design space of reinforcement learning for evolutionary algorithms and highlights online control as a key direction for scalable adaptive search [10]. More recently, Wang et al. propose an MRL-MOEA framework based on multi-role learning in a many-objective

Table 1. Compact Comparative Landscape (Baselines and Adaptive MOEA Positioning)

Method / family	Many-obj.	Online learning	Adaptivity	Relevance to this study
NSGA-II [3]	Med.	No	No	Standard baseline for multi-objective optimization; included for methodological positioning.
NSGA-III [6]	High	No	No	Direct paired baseline in the artifact; strong selection, though operator schedule remains static.
MOEA/D [4]	High	No	No	Strong decomposition baseline; included for comparative positioning.
Recent adaptive MOEAs [9,11,12]	High	Yes	Yes	Learning-guided control in generic MOEA settings; limited workflow-specific evidence.
ADARE (this work)	High	Yes (UCB)	Yes	Workflow-specific learning-guided adaptive control with fairness-controlled evaluation.

setting [11], and Xue et al. report a Q-learning-assisted MOEA/D variant with adaptive weight adjustment [12].

These studies support the broader premise of *learning-guided evolutionary control*. In our setting, the key distinction is not only whether learning is used, but where it is inserted in the optimization loop and what runtime cost it introduces. Most published results target generic benchmark suites rather than workflow scheduling with heterogeneous costs and bandwidth asymmetry. ADARE addresses that systems-specific gap while keeping a controller that can be profiled directly on a local workstation.

2.4 Comparative Landscape of Cited Baselines

Table 1 summarizes the roles of the most relevant baselines and adaptive MOEA families discussed above. The table is intentionally split between *algorithmic positioning* and *artifact availability*: in the current code artifact, NSGA-III is the implemented baseline for direct paired comparison, while NSGA-II and MOEA/D are included as methodological baselines for broader positioning.

With that positioning in place, we can state the scheduling model and the objective vector actually used by the online control loop.

3 Problem Modeling

3.1 System Model

We consider a workflow represented as a directed acyclic graph (DAG) $G = (V, E)$, where each task $v_i \in V$ carries a computational load and a data transfer

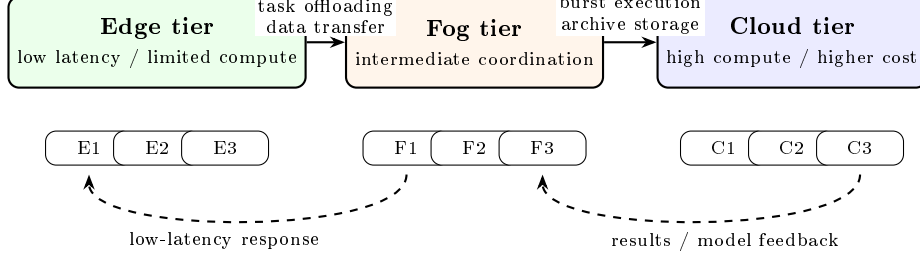


Fig. 1. Simplified Edge/Fog/Cloud execution architecture used to motivate the scheduling model. The heterogeneous compute, bandwidth, and cost properties across tiers create the many-objective trade-offs optimized by ADARE and NSGA-III.

footprint, and each directed edge encodes a precedence relation with communication dependency. The execution platform is a heterogeneous resource set R spanning edge, fog, and cloud nodes, each characterized by processing rate, communication bandwidth, monetary processing cost, and power usage.

A candidate schedule is encoded as a task-to-node assignment vector

$$x = (x_1, \dots, x_{|V|}), \quad x_i \in \{0, 1, \dots, |R| - 1\}. \quad (1)$$

Given a topological order of the DAG, the evaluator simulates task start and finish times under node availability and inter-node transfer delays.

Figure 1 clarifies the systems setting: edge nodes favor low-latency execution, fog nodes provide coordination and intermediate capacity, and cloud nodes provide stronger compute at a different cost/bandwidth profile. This heterogeneity is the reason a static operator schedule can become suboptimal during search.

3.2 Objective Vector

ADARE and NSGA-III optimize the same four-objective minimization vector:

$$\min \mathbf{f}(x) = (f_{\text{mk}}(x), f_{\text{lat}}(x), f_{\text{cost}}(x), f_{\text{eng}}(x)), \quad (2)$$

where:

- f_{mk} is the workflow makespan (maximum task completion time);
- f_{lat} aggregates task completion delays relative to readiness times;
- f_{cost} accumulates compute-time-weighted processing cost across assigned nodes;
- f_{eng} accumulates compute-time-weighted working power as an execution energy proxy.

In our runs, we repeatedly observed this tension in edge/fog/cloud orchestration. Faster assignments reduce completion time. Sometimes sharply. Depending on tier placement, cost or energy can then increase. So we target a diverse, high-quality Pareto set rather than a single operating point.

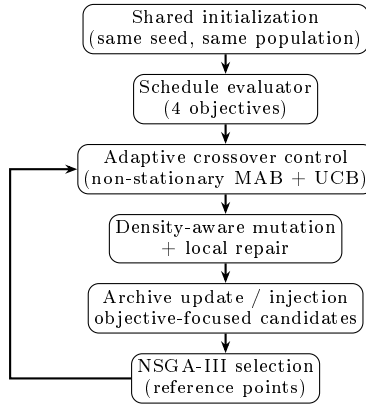


Fig. 2. ADARE as a learning-guided control layer embedded in the NSGA-III search loop.

3.3 Evaluation Targets and Quality Indicators

In addition to objective values, we evaluate search quality using hypervolume (HV), inverted generational distance (IGD), spacing, additive epsilon, and coverage. No single indicator was reliable enough on its own in our runs, so we keep this set to read front quality from complementary angles (dominance coverage, convergence to a reference front, and spread regularity). For ADARE, that distinction matters: the controller is meant to improve search behavior over time, not just one objective extreme.

With the objective vector and evaluation targets fixed, the remaining question is how the search loop uses this feedback online. We next describe the operator controller and the diversity/archive mechanisms that were retained after implementation and profiling.

4 ADARE Methodology: Learning-Guided Adaptive Control

4.1 Design Overview

ADARE keeps the many-objective selection backbone of NSGA-III and replaces fixed operator control with an online adaptive layer. Figure 2 summarizes this organization.

The architectural decision was deliberate. Replacing the full optimizer with a monolithic learner would make paired comparisons harder to interpret and would blur where the gains come from. We therefore attach the adaptive components to operator choice and diversity recovery while leaving NSGA-III selection unchanged; this keeps the method interpretable and preserves a runtime profile that is still practical on a local workstation.

Algorithm 1 ADARE UCB-Based Crossover Control**Require:** Parent pair (p_1, p_2) , scores Q , usage counts N , step t

- 1: Select operator index k with Eq. (3)
- 2: Apply crossover operator k to obtain (c_1, c_2)
- 3: Compute reward r_t with Eq. (4)
- 4: Update $Q_k \leftarrow (1 - \alpha)Q_k + \alpha r_t$ and $N_k \leftarrow N_k + 1$
- 5: **return** (c_1, c_2)

4.2 Learning Formalization: Non-stationary MAB for Operator Control

At each generation, ADARE selects a crossover operator from a small portfolio (one-point, two-point, and uniform variants). We model this choice as a multi-armed bandit (MAB) problem. The formulation is intentionally local: a compact controller is easier to instrument, debug, and compare fairly than a higher-capacity policy module in this artifact setting. The reward distribution is non-stationary because population diversity, front geometry, and feasibility structure evolve during search [7,8], which makes static crossover probabilities a poor fit for heterogeneous workflow scheduling.

Let Q_k be the current quality estimate for operator k , and N_k its usage count. At decision step t , ADARE chooses the operator with the UCB rule:

$$a_t = \arg \max_k \left(Q_k + \sqrt{\frac{2 \ln t}{N_k + 1}} \right). \quad (3)$$

The second term maintains exploration, while the first term exploits operators with strong recent rewards.

Reward definition. We define the reward as a scalar Pareto-quality improvement proxy computed on the parent-offspring comparison:

$$r_t = \phi(\min\{p_1, p_2\}) - \phi(\min\{c_1, c_2\}), \quad (4)$$

where $\phi(\cdot)$ is a log-scaled scalarization proxy (smaller is better). We retained this proxy after a few implementation attempts because front-level indicators at every mating event made operator-level attribution both expensive and noisy. A positive reward indicates that the selected operator produced better offspring according to the proxy. ADARE updates the selected operator value by exponential smoothing:

$$Q_k \leftarrow (1 - \alpha)Q_k + \alpha r_t. \quad (5)$$

This keeps operator control in an online-learning regime, while the update itself remains cheap enough for clean overhead measurements.

4.3 Density-Aware Mutation and Local Repair

ADARE’s mutation layer adapts perturbation effort using generation progress and population diversity. When diversity collapses, mutation pressure increases;

Algorithm 2 Density-Aware Mutation with Heuristic Guidance

Require: Individual x , diversity score d , generation g , task-node rankings $\mathcal{R}$

- 1: Compute mutation budget $B \leftarrow f(g, d)$ (larger when diversity decreases)
- 2: **for** $b = 1$ to B **do**
- 3: Sample a task index i
- 4: Set x_i using heuristic-ranked nodes or a random fallback
- 5: **end for**
- 6: Apply a low-probability gene-level mutation pass
- 7: Optionally perform a small greedy local repair
- 8: **return** mutated individual x

Algorithm 3 Archive-Guided Final Objective Pool Construction

Require: Final population P , archive A , targeted candidate set K

- 1: $P^{\text{obj}} \leftarrow P \cup A$
- 2: **for all** $k \in K$ **do**
- 3: **if** k improves at least one tracked objective extreme **then**
- 4: Insert k into P^{obj}
- 5: **end if**
- 6: **end for**
- 7: **return** P for Pareto-quality metrics and P^{obj} for objective-best metrics

when diversity is healthy, mutation stays conservative. A heuristic task-node ranking (processing speed, bandwidth, cost, power) biases part of the mutations toward plausible assignments, but we retain a random fallback to avoid over-committing to the heuristic. This balance matters in workflow scheduling because naive perturbations can break good structures under precedence and transfer constraints, while purely heuristic moves can reduce exploration too early.

4.4 Archive Guidance and Objective-Focused Final Candidate Pool

ADARE maintains a bounded archive of strong non-dominated or high-value candidates. During evolution, archive injection is rate-limited so it improves search guidance without collapsing diversity. At the end of the run, ADARE separates two roles: (i) the final non-dominated population for Pareto-quality metrics (HV/IGD/spacing/epsilon/coverage), and (ii) an objective-focused pool augmented with archive and targeted candidates for best-objective reporting (e.g., makespan or latency extremes). This separation is important for interpretation: otherwise, very specialized archived candidates can improve best-objective reporting while giving a misleading impression of the final Pareto front quality.

4.5 Complexity and Measured Control Overhead

The schedule evaluator dominates runtime. For population size P , generation count G , task count $|V|$, and average dependency-processing factor $\bar{d}$, evaluation cost is approximately

$$\mathcal{O}(G \cdot P \cdot |V| \cdot \bar{d}). \quad (6)$$

The UCB controller adds constant-time operations per operator decision (mean update, counter update, logarithm, square root), while archive operations are

Table 2. Local Host Machine (from `data/dxdiag.txt`)

Item	Specification
System model	ASUS TUF Gaming A15 FA506NF_FA506NF (ASUSTeK COMPUTER INC.)
Host OS	Windows 11 Home 64-bit, Build 26200 (dxdiag timestamp: 2025-12-18)
CPU	AMD Ryzen 5 7535HS with Radeon Graphics, 12 logical CPUs, 3.300 GHz nominal
RAM	16.000 GB installed (15.690 GB available in dxdiag)
GPU(s)	AMD Radeon(TM) Graphics + NVIDIA GeForce RTX 2050 (3.964 GB dedicated reported)
Display / mode	1920×1080; host reports 144 Hz internal panel and 75 Hz external display

bounded on a small structure. In this artifact, the schedule evaluator still dominates wall-clock time for realistic workflow sizes.

To verify this empirically, we instrumented UCB selection and update, together with archive routines, on 15 representative runs (5 seeds per benchmark). The measured upper-bound control ratio is **5.17% on average** (maximum **5.34%**) of total ADARE runtime. The instrumentation was intentionally conservative at Python level, so the ratio should be read as an upper bound rather than an exact decomposition of every micro-operation. It nevertheless supports the intended near-5% overhead target while preserving measurable quality gains.

5 Experimental Protocol and Results

5.1 Artifact, Local Execution Context, and Hardware Configuration

All experiments were executed locally using the code in the current ADARE artifact (no cluster). We report the host details from `data/dxdiag.txt` because runtime differences at this scale are sensitive to workstation-level CPU and memory behavior.

Table 2 is operationally important because runtime gains in Python MOEAs depend on CPU generation and memory behavior. This became especially relevant when some longer runs were split into batches for execution stability. The local software stack used for the experiments is Python 3.13.1 with `numpy` 2.2.5, `matplotlib` 3.10.8, `deap` 1.4, `scipy` 1.16.3, and `pymoo` 0.6.1.6.

5.2 Benchmarks and Heterogeneous Resource Configuration

We evaluate three workflow instances commonly used in scientific workflow scheduling: `Montage_25`, `CyberShake_30`, and `Epigenomics_24`. These workflow families are available through public repositories used in the Pegasus and Wf-Commons ecosystems [5].

The heterogeneous resource model is defined by `data/environments.json`. Table 3 reproduces the exact tier-level parameters used in the artifact.

Table 3. Resource Configuration from `data/environments.json`

Tier	Devices	Proc.	rate	Cost	Idle P.	Work P.	Up/Down	BW
Edge	5	1000	0.02	30	700	20480	/	20480
Fog	5	1300	0.48	30	700	10000	/	10000
Cloud	5	1600	0.96	1332	1648	100	/	10000

Table 4. 20-Run Summary (ADARE vs NSGA-III, means; NSGA-III in parentheses)

Benchmark	HV	IGD	Δ Mk. (%)	Δ Lat. (%)	Δt (%)
Montage_25	0.9493 (0.9232)	0.0600 (0.0624)	19.01	11.77	4.89
CyberShake_30	0.9645 (0.9303)	0.0633 (0.0718)	5.17	8.01	7.47
Epigenomics_24	1.0453 (1.0337)	0.0342 (0.0414)	1.18	4.34	4.13

This table creates the optimization tension used throughout the study. In early debugging runs, it was easy to over-focus on cloud processing rate alone; the bandwidth asymmetry and cost/power gradients explain why that intuition is often wrong.

5.3 Fairness Protocol, Metrics, and Statistical Testing

ADARE and NSGA-III share the same evaluation function, generation budget, and initial population at each paired run. The artifact generates a shared initial population from the same seed before instantiating both algorithms, which enforces a fair comparison baseline.

We report **20 paired runs per benchmark**. In our runs on the workstation, benchmarks are executed sequentially (one workflow family at a time) to avoid runtime perturbations from concurrent processes. The statistical design is unchanged: paired 20-run comparisons.

Reported outputs include: (i) Pareto-quality metrics (HV, IGD, spacing, epsilon, coverage), (ii) runtime, and (iii) best objective values for makespan, latency, cost, and energy. For statistical validation, we use the paired Wilcoxon signed-rank test and Cliff’s delta effect size [15,16].

5.4 Main Results: 20-Run Paired Comparison (ADARE vs NSGA-III)

Table 4 summarizes the main comparative outcomes (means over 20 paired runs). NSGA-III means are shown in parentheses for direct readability.

ADARE improves HV on all three workflows and improves makespan/latency extremes in every case. In the final 20-run configuration, IGD also improves on all three workflows. This is useful because it means the stronger extremes and spread do not come at the expense of distance-to-reference quality in the reported protocol.

Table 5 confirms that the quality gains are not only visual. HV improvements are significant on all three workflows, and the selected runtime comparisons

Table 5. Statistical Evidence (20 Paired Runs): Wilcoxon p and Cliff’s δ

Benchmark	Metric	Wilcoxon p	Cliff’s δ
Montage_25	HV	1.05×10^{-4}	0.645 (large)
	Coverage	1.33×10^{-2}	0.440 (medium)
	Runtime	6.04×10^{-3}	0.725 (large)
CyberShake_30	HV	1.91×10^{-6}	0.980 (large)
	IGD	9.54×10^{-7}	0.940 (large)
	Runtime	1.97×10^{-4}	0.760 (large)
Epigenomics_24	HV	9.54×10^{-7}	0.960 (large)
	Spacing	9.54×10^{-7}	0.965 (large)
	Runtime	2.66×10^{-2}	0.325 (small)

Table 6. Ablation Variants (Design Matrix for the 20-Run Protocol)

Variant	UCB crossover	Density-aware mutation	Archive/objective pool
V1 (NSGA-III)	No	No	No
V2	Yes	No	No
V3	Yes	Yes	No
V4	Yes	Yes	Yes (partial)
V5 (ADARE full)	Yes	Yes	Yes (full config)

are also significant in this final protocol. Epigenomics nevertheless remains the benchmark where runtime gains are the smallest in effect size (small Cliff’s δ), even when the direction is favorable.

In our runs across the three benchmarks, ADARE wins **18/18** quality comparisons (including runtime) and **15/15** core-quality comparisons (excluding runtime). Average gains are **26.80%** in core quality, **10.70%** in best-objective metrics, and **5.50%** in runtime.

5.5 Ablation Protocol (V1–V5) and Artifact-Ready Variant Definition

To isolate module effects in our runs, we keep the same paired-run fairness constraints and grow the ablation step by step. We start with **V1** (NSGA-III baseline), then add UCB crossover control in **V2**, density-aware mutation in **V3**, partial archive injection/final-pool guidance in **V4**, and full ADARE in **V5**.

This paper reports the completed 20-run ADARE-vs-NSGA-III comparison and provides the V1–V5 ablation protocol and variant mapping for the extended artifact. Methodologically, the gains are better interpreted as the interaction of several control components than as the effect of a single heuristic switch.

5.6 Qualitative Pareto and Convergence Analysis

We inspect representative Pareto projections and convergence traces next because the tables alone do not show *how* the search behavior changes.

Figure 3 is consistent with the quantitative results: ADARE extends favorable regions on Montage, broadens coverage on CyberShake, and improves spread on

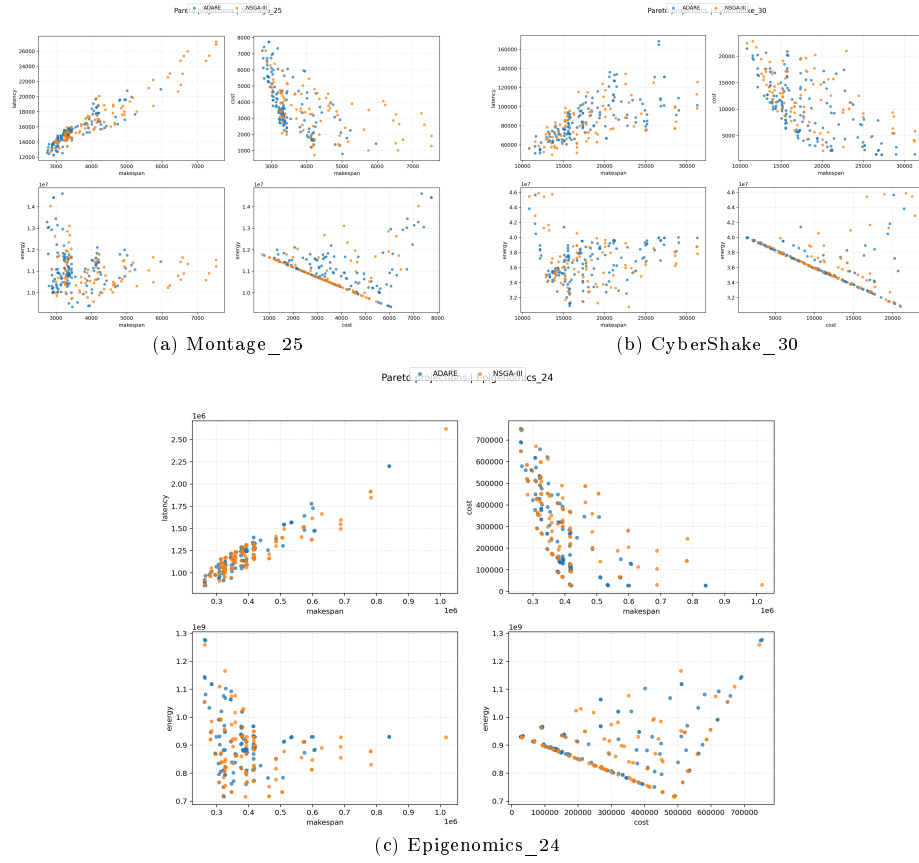


Fig. 3. Representative Pareto projections (ADARE vs NSGA-III) across the three workflow families. Larger panels are used here for readability of the projected fronts and dominance regions.

Epigenomics while preserving strong objective extremes. The Epigenomics plots are particularly useful here because the visual spread improvement is easier to see than the runtime effect.

Figure 4 makes the control effect visible in our runs. This matters. We see earlier trajectory separation on Montage and CyberShake, and later generations stabilize differently. Epigenomics still improves, mainly through front structure, while runtime movement stays smaller.

At this point, the practical question is less “does ADARE ever help?” and more “when does the adaptive layer help most, and where are its limits?” The discussion below addresses that question directly.

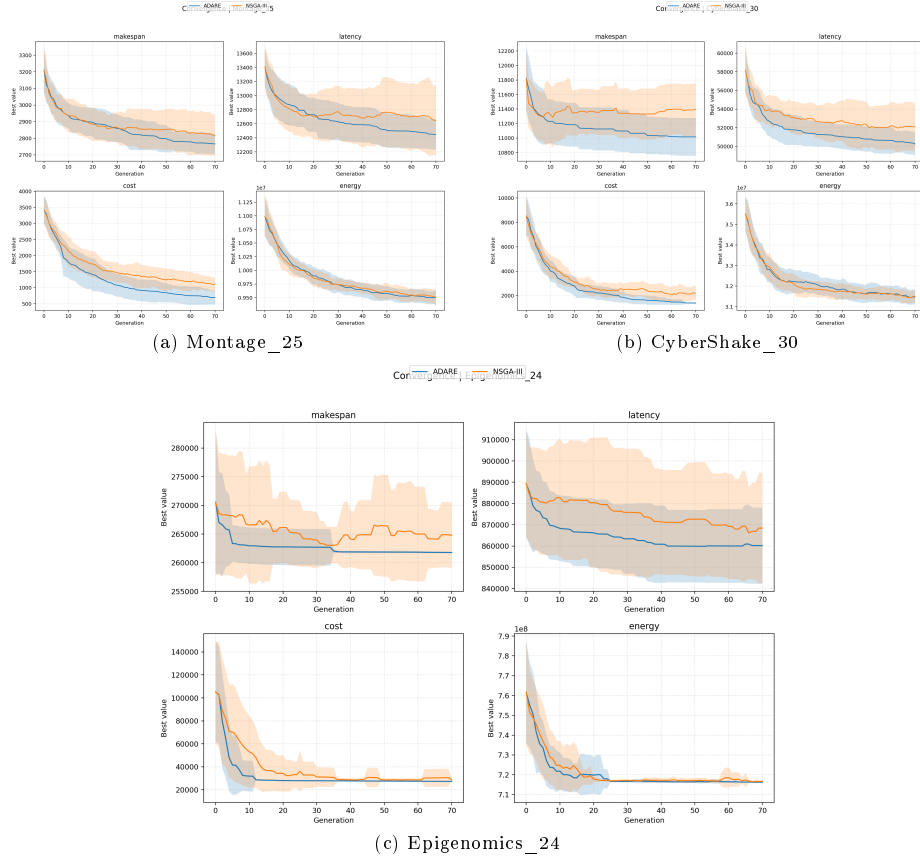


Fig. 4. Convergence traces (best objective values by generation) for Montage, CyberShake, and Epigenomics. Larger panels are used to improve readability of generation-wise trajectories and confidence bands.

6 Discussion

6.1 Why Static NSGA-III Underperforms in Heterogeneous Environments

In our setting, the main weakness of a static NSGA-III pipeline is not the selection operator itself; NSGA-III’s reference-point mechanism remains a strong many-objective baseline. The recurring issue is *operator control rigidity*. Fixed crossover and mutation schedules assume a reward landscape that stays roughly stable, but workflow scheduling across heterogeneous edge, fog, and cloud resources shows phase changes during search.

Early generations benefit from broad exploration to discover useful task-node assignments under precedence and bandwidth constraints. Later generations benefit more from operators that preserve partial structure and refine local trade-offs. A static schedule cannot represent this shift explicitly, while ADARE’s controller adapts operator preference from observed rewards. The convergence

plots were useful here because they exposed this phase dependence before the aggregate statistics made it obvious.

6.2 Justification of Experimental Choices

Population size and generation count. We use population size 100 and 70 generations to keep comparisons computationally affordable on a local workstation while preserving enough diversity for four-objective optimization. Increasing population size improves front coverage but raises evaluator cost in practice through more offspring evaluations and more frequent archive maintenance. The chosen configuration is a compromise between statistical replication and per-run search depth.

Seeding and fairness. Seed control is not a cosmetic choice. In stochastic MOEAs, comparing unmatched runs can confound algorithmic differences with initialization variance. Our paired design uses the same initial population for ADARE and NSGA-III for every run, which makes Wilcoxon and Cliff’s delta more informative.

Why 20 runs. The original artifact analysis used 6 runs. In our runs, that count exposed strong effects but remained fragile for smaller effect-size comparisons, especially runtime relative to HV/IGD/spacing gains. We moved to 20 paired runs to stabilize mean estimates and effect sizes. Table 5 shows that directions remain favorable while effect sizes still vary by benchmark and metric.

6.3 Sensitivity and Robustness Trends

Our observations suggest three sensitivity patterns.

1. **Run count sensitivity:** metrics with smaller effect sizes (most clearly runtime on Epigenomics in our final protocol) are sensitive to the number of repetitions and should not be interpreted from very small samples.
2. **Mutation/Archive sensitivity:** ADARE’s strongest empirical gains are associated with spacing, coverage, and objective extremes, indicating that density-aware mutation and archive guidance dominate the observed improvements in difficult landscapes.
3. **Benchmark dependence:** ADARE delivers strong quality gains on all three workflows, but quality and runtime do not move with the same amplitude on every instance. Epigenomics shows a smaller runtime effect size than CyberShake or Montage despite clear quality improvements.

These patterns support the adaptive-control interpretation in our runs. Under the reported 20-run protocol, ADARE is superior on all tracked quality metrics, yet the *magnitude* of improvement still depends on benchmark structure. For that reason, we avoid benchmark-independent effect-size claims from smaller repetition counts, even when win/loss counts look unambiguous.

6.4 Limitations and Artifact Scope

Two limitations should be stated explicitly. First, the current artifact directly implements NSGA-III as the paired baseline; NSGA-II and MOEA/D are included in our comparative positioning but not yet re-implemented under the same evaluator in this code snapshot. Second, while we provide the required V1–V5 ablation protocol and variant mapping, the full 20-run ablation sweep is deferred to a dedicated artifact extension because it substantially expands total runtime on a local workstation.

These limitations do not invalidate the main ADARE-vs-NSGA-III comparison, but they narrow the scope of what can be concluded from this version of the artifact. In particular, the broader baseline and ablation story is a reproducibility/engineering completion task rather than a theoretical gap.

7 Conclusion

We reframed ADARE as an online control layer for many-objective workflow scheduling rather than as a simple scheduler variant. This framing is technically useful because it explains why operator-level learning is central to the method, while the evolutionary backbone can remain largely unchanged.

Under a fairness-controlled protocol (same initialization, same budget, same evaluator) and a stronger statistical setup (20 paired runs per benchmark), ADARE improves overall many-objective search quality and runtime against NSGA-III on three heterogeneous workflow instances. In our runs, gains are strongest in HV, coverage, spacing, and objective extremes, and CyberShake shows the clearest combined benefit. Runtime also improves on all three benchmarks in the final protocol, although the Epigenomics runtime effect remains smaller than its quality effects. Measured control overhead stays near 5% on average, preserving practical usability on a local workstation.

For the ECML PKDD framing, the main takeaway is that workflow scheduling in heterogeneous distributed systems benefits from treating evolutionary operator choice as an *online learning control problem*. Future work will complete the 20-run V1–V5 ablation sweep and extend the artifact with directly implemented NSGA-II and MOEA/D baselines under the same evaluator.

References

1. Topcuoglu, H., Hariri, S., Wu, M.-Y.: Performance-effective and low-complexity task scheduling for heterogeneous computing. *IEEE Transactions on Parallel and Distributed Systems* **13**(3), 260–274 (2002)
2. Bonomi, F., Milito, R., Zhu, J., Addepalli, S.: Fog computing and its role in the Internet of Things. In: *MCC Workshop on Mobile Cloud Computing*, pp. 13–16 (2012)
3. Deb, K., Pratap, A., Agarwal, S., Meyarivan, T.: A fast and elitist multiobjective genetic algorithm: NSGA-II. *IEEE Transactions on Evolutionary Computation* **6**(2), 182–197 (2002)
4. Zhang, Q., Li, H.: MOEA/D: A multiobjective evolutionary algorithm based on decomposition. *IEEE Transactions on Evolutionary Computation* **11**(6), 712–731 (2007)

5. Coleman, T., Babuji, Y., Li, H., et al.: WfCommons: A framework for enabling scientific workflow research and development. *Scientific Data* **9**, 220 (2022).
6. Deb, K., Jain, H.: An evolutionary many-objective optimization algorithm using reference-point-based nondominated sorting approach, part I: Solving problems with box constraints. *IEEE Transactions on Evolutionary Computation* **18**(4), 577–601 (2014)
7. Auer, P., Cesa-Bianchi, N., Fischer, P.: Finite-time analysis of the multiarmed bandit problem. *Machine Learning* **47**, 235–256 (2002)
8. Besbes, O., Gur, Y., Zeevi, A.: Stochastic multi-armed-bandit problem with non-stationary rewards. In: *Advances in Neural Information Processing Systems* (NeurIPS), vol. 27 (2014)
9. Jiao, K., Chen, J., Xin, B., Li, L.: OVEA: A reference vector based multiobjective evolutionary algorithm with Q-learning for operator adaptation. *Evolutionary Computation* **31**(3), 375–400 (2023)
10. Song, A., Wang, J., Li, K., Zhang, Q.: Reinforcement learning-assisted evolutionary algorithm: A survey and research opportunities. *ACM Computing Surveys* **56**(12), Article 311 (2024)
11. Wang, K., Wang, Y., Xie, X., Yu, S., Zhang, Q.: MRL-MOEA: Multi-role learning based many-objective evolutionary algorithm. *Scientific Reports* **15**, 25039 (2025)
12. Xue, J., Yang, S., Zhao, X., Hu, Y.: QLMOEA/D-AWA: A Q-learning-based decomposition multi-objective optimization evolutionary algorithm with adaptive weight adjustment. *Scientific Reports* **15**, 17290 (2025)
13. Cui, W., Song, A., Wang, J., Zhang, Q.: DRLMMEA: Deep reinforcement learning-assisted many-objective evolutionary algorithm. *Nature Communications* **17**, 1084 (2026)
14. Mao, Y., You, C., Zhang, J., Huang, K., Letaief, K.B.: A survey on mobile edge computing: The communication perspective. *IEEE Communications Surveys & Tutorials* **19**(4), 2322–2358 (2017)
15. Wilcoxon, F.: Individual comparisons by ranking methods. *Biometrics Bulletin* **1**(6), 80–83 (1945)
16. Cliff, N.: Dominance statistics: Ordinal analyses to answer ordinal questions. *Psychological Bulletin* **114**(3), 494–509 (1993)
17. Karafotias, G., Hoogendoorn, M., Eiben, A.E.: Parameter control in evolutionary algorithms: Trends and challenges. *IEEE Transactions on Evolutionary Computation* **19**(2), 167–187 (2015)
18. Coello Coello, C.A., Lamont, G.B., Van Veldhuizen, D.A.: *Evolutionary Algorithms for Solving Multi-Objective Problems*, 2nd edn. Springer (2007)
19. Deelman, E., Singh, G., Su, M.-H., et al.: Pegasus: A framework for mapping complex scientific workflows onto distributed systems. *Scientific Programming* **13**(3), 219–237 (2005)
20. WfCommons Project: WfCommons: benchmark instances and workflow tools. <https://wfcommons.org/> (accessed February 24, 2026)